# Project 2: Key-Value Store

*Due: 11:59 PM, April 9*

## Project Goal

In this project, you will rapidly deploy, configure, and evaluate a distributed **key-value store** using Redis Cluster on a cloud platform. The focus is on understanding and implementing essential concepts for scalable key-value storage: sharding for data distribution, replication for durability/read scaling, high availability through failover, and basic performance testing under typical key-value workloads.

## Project Tasks

1. **Cloud Environment Setup:**
    - Select a cloud provider (AWS, GCP, Azure).
    - Provision **3-6** virtual machines or container instances.
    - Configure basic private networking and necessary security group rules to allow Redis communication between nodes. (*Manual setup is acceptable; simple scripts or basic IaC templates are encouraged.*)
    - Document the cloud resource setup.
2. **Basic Redis Cluster Deployment (as Key-Value Store):**
    - Install Redis on all provisioned nodes.
    - Use `redis-cli --cluster create` (or an equiv
      Aim for at least 3 master nodes and configure a

masters, 3 replicas for a 6-node setup) to ensure **key durability and read availability**.
- Verify the cluster status ( `cluster info` , `cluster nodes` ) and ensure all nodes are correctly joined and hash slots (determining key placement) are assigned.
- Document the key configuration steps and final Redis configuration files.

3. **Failover Configuration and Testing (for Key Availability):**

- Confirm that Redis Cluster is configured for automatic failover to maintain **continuous access to keys**.
- Perform a failover test: Manually stop one of the master nodes responsible for a subset of keys.
- Observe the cluster's reaction using `cluster nodes` or monitoring logs. Verify that a replica is promoted to master status, taking over responsibility for its assigned keys.
- Check that the cluster remains operational for clients (key reads/writes continue, possibly after a brief pause).
- Restart the stopped node and verify it rejoins the cluster as a replica.
- Document the failover test procedure and observations regarding key availability during the event.

4. **Key-Value Workload Generator Implementation:**

- Develop or adapt a **simple** workload generator program focusing on simulating **key-value access patterns**.
- Ensure the generator can connect to the Redis cluster (using a cluster-aware client library is strongly recommended).
- Implement logic primarily performing high-frequency, basic key operations like `SET` , `GET` , and potentially `DEL` or `INCR` , targeting random keys to distribute the load across shards. *Usage of more complex data structures should be minimal unless justified for representing structured values.*
- Incorporate concurrency (e.g., using multiple threads, asynchronous I/O) to simulate multiple clients accessing keys simultaneously.
- Add basic functionality to measure **latency per key operation** and calculate overall **throughput** (key operations per second).

5. **Performance Evaluation (Key-Value Focus):**

- Execute the workload generator against the stable cluster to establish baseline **key operation performance** (latency, throughput). Record these results.
- Run the workload generator again, and *during* th
master node).

- Observe and record the impact on **key access performance** (latency spikes, throughput drops) during the failover.
- Measure or estimate the time it takes for the cluster to stabilize and performance to recover for key operations after the failover completes.

6. **Reporting and Finalization:**

- Analyze the collected performance data and failover observations, focusing on the system's behavior as a distributed key-value store.
- Write the concise Evaluation Summary Report, including architecture, workload (key access patterns), results (key operation performance), failover analysis, and challenges.
- Organize all source code (workload generator, any scripts), configuration files, and the final report into a repository for submission. Ensure code and configurations are reasonably documented or self-explanatory.

## Graduate Extension

**Undergraduate students may attempt this section for extra credit (up to +10 points). Graduate students are REQUIRED to complete this section and it is worth 10% of the total project grade.**

7. **Graduate Component: Elasticity or Recovery Extension**
   - Extend your Redis-based key-value store with **one** meaningful advanced capability beyond the core requirements.
   - Acceptable options include:
     - **Elastic scaling / resharding:** Add or remove a node and rebalance hash slots, then measure the effect on availability and performance.
     - **Persistence and recovery:** Enable and evaluate Redis persistence (RDB or AOF), then demonstrate data recovery after node restart or simulated failure.
     - **Read-scaling experiment:** Compare read-heavy workload behavior before and after using replicas for read traffic.
     - **Cloud automation:** Replace manual deployment with repeatable automation (Terraform, Ansible, deployment scripts, or equivalent) and demonstrate reproducible cluster creation.
   - Your extension must include:
     - A clear description of the added capability.
     - The commands, configuration, or code needed to implement it.
     - Evidence that it worked.
     - A short analysis of the tradeoffs, benefits, a

## Core Deliverables

1. **Cloud Setup Description:** Brief documentation or annotated scripts outlining the cloud resources (VMs/containers, networking basics, security groups) used. Full IaC is *optional* but encouraged if feasible.
2. **Deployed Redis Cluster Configuration:** Key configuration files and commands used to set up the sharded and replicated Redis cluster, optimized for key-value storage.
3. **Workload Generator:** Source code for a workload simulation tool focusing on **key-value operations** (e.g., high frequency `GET`, `SET`, `DEL`, possibly simple `HSET` / `HGET`).
4. **Evaluation Summary Report:** A report (no page limit) detailing:
   - The final cluster architecture for the key-value store.
   - Description of the simulated key-value workload.
   - Key performance results (**key operation latency**, **throughput** graphs).
   - Observations during failover testing (impact on key access, recovery time).
   - Brief discussion of challenges encountered.
5. **Source Code & Config Files:** Repository containing deployment scripts (if any), Redis configs, workload generator code.
6. **Graduate Extension Evidence (Graduate Students Required / Undergraduate Extra Credit):** Documentation, code/configuration, and results for the advanced extension described above.

## Submission Guidelines

**Format:** Submit a **SINGLE PDF** file named `Project2_YourName.pdf` to Brightspace. Include your name on the first page and clearly indicate whether you are submitting as an undergraduate or graduate student.

**Report Requirements:** Your report is the primary evidence of your work. For **EACH** task you complete, include:

1. **Commands Run:** The exact CLI commands you executed.
2. **Output / Evidence:** Relevant command output, logs, screenshots, or plots showing the system behavior.
3. **Configuration / Code:** Copy and paste the key configuration files, workload generator code, and any automation scripts directly into the PDF.
4. **Explanation:** A brief paragraph explaining why you configured the system that way and what each step demonstrates.
5. **Analysis:** For the performance and failover tasks, briefly interpret the observed latency, throughput, and recovery behavior.

6. **Graduate Component (if attempted or required):** Include the implementation details, evidence, and a short discussion of results and tradeoffs.

Submit all deliverables in a single PDF file (including code and configuration files), and upload it on Brightspace.

## Grading Scheme

The project is graded out of **100 points**.

| Category | Points | Criteria |
|---|---|---|
| **Documentation & Evidence** | **25** | **Did you show your work?** Full points if the report includes commands, outputs, screenshots, and configuration evidence for each major task. Deductions for vague descriptions, missing logs, or incomplete evidence. |
| **Technical Correctness** | **25** | **Is the Redis deployment correct?** Full points for a functional Redis Cluster with correct sharding, replication, and network/security configuration. Deductions for broken cluster membership, incorrect slot assignment, or misconfigured replication/failover behavior. |
| **Failover Testing & Analysis** | **20** | **Did you meaningfully test resilience?** Points awarded for correctly executing failover experiments, showing service continuity, and analyzing recovery behavior and system impact. |
| **Workload Generator & Performance Evaluation** | **20** | **Did you build and use a suitable benchmark?** Full points for a working workload generator that measures latency and throughput under concurrent key-value operations, plus clear baseline and failover performance results. |
| **Graduate Requirement / Undergraduate Bonus** | **10** | **(Required for Graduate Students / Extra Credit for Undergraduates)** Successful comp beyond the core p |

Chat Assistant

| Category | Points | Criteria |
| --- | --- | --- |
| | | and analysis.<br>*Note: Graduate students are graded out of 100 including this section. Undergraduates who do not attempt this section are graded on the 90 core points, and successful completion earns up to +10 bonus points.* |

You are expected to primarily rely on official online documentation to research and understand how to complete each project task. Key resources include the Redis official documentation (https://redis.io/documentation), the documentation for your chosen cloud provider (AWS, GCP, or Azure), and potentially documentation for relevant tools or libraries (like redis-py if used).

You are ALLOWED to use LLM-based AI assistants (e.g., ChatGPT, GitHub Copilot) as a supplementary tool. These can be helpful for generating boilerplate code, debugging specific errors, explaining configuration options, or exploring alternative approaches. However, you are responsible for validating the correctness, security, and applicability of any information or code provided by an LLM. Directly submitting unverified or misunderstood LLM output is not acceptable. You must be able to explain all parts of your final submission.

**Chat Assistant**